DAY 02 OF 16 · 3D & 6DOF STREAMING

# Coordinates & transformations

How to move, rotate and place 3D data, and how a 6DoF pose becomes one 4×4 matrix.

Points & frames | Translation | Rotation | 4×4 matrices | Order matters | Quaternions

Book: Ch. 2 (mathematics) · Code: day02_coordinates_transforms/

THE BIG IDEA

# Every 3D operation in this course is: multiply the points by a matrix.

Moving a scan, placing a camera, aligning two scans, rendering a view: all the same tool.

COORDINATE SYSTEMS

# A point only has coordinates relative to a frame

## World frame

Fixed reference for the scene. Our Python code: Z is up, units are metres.

01

## Object frame

Attached to a thing: the car's front is its own +x.

02

## Camera frame

Attached to the viewer. OpenCV: +z forward, x right, y down. three.js: −z forward, y up.

03

TRANSLATION AND ROTATION

# Moving = adding a vector; turning = multiplying by a matrix

$$p' = R \cdot p + t$$

### p

The point (x, y, z) we start with.

### R

3×3 rotation matrix. Columns are the new axes. $R^T R = I$, $\det R = 1$.

### t

Translation vector: how far to shift in x, y, z.

HOMOGENEOUS COORDINATES

# Pack R and t into one 4×4 matrix

$$[x'\ y'\ z'\ 1]^T = \begin{bmatrix} R & t \\ 0\ 0\ 0 & 1 \end{bmatrix} \cdot [x\ y\ z\ 1]^T$$

### Why the extra 1?

It lets translation be a multiplication too, so ANY chain of moves is one matrix product.

### Chaining

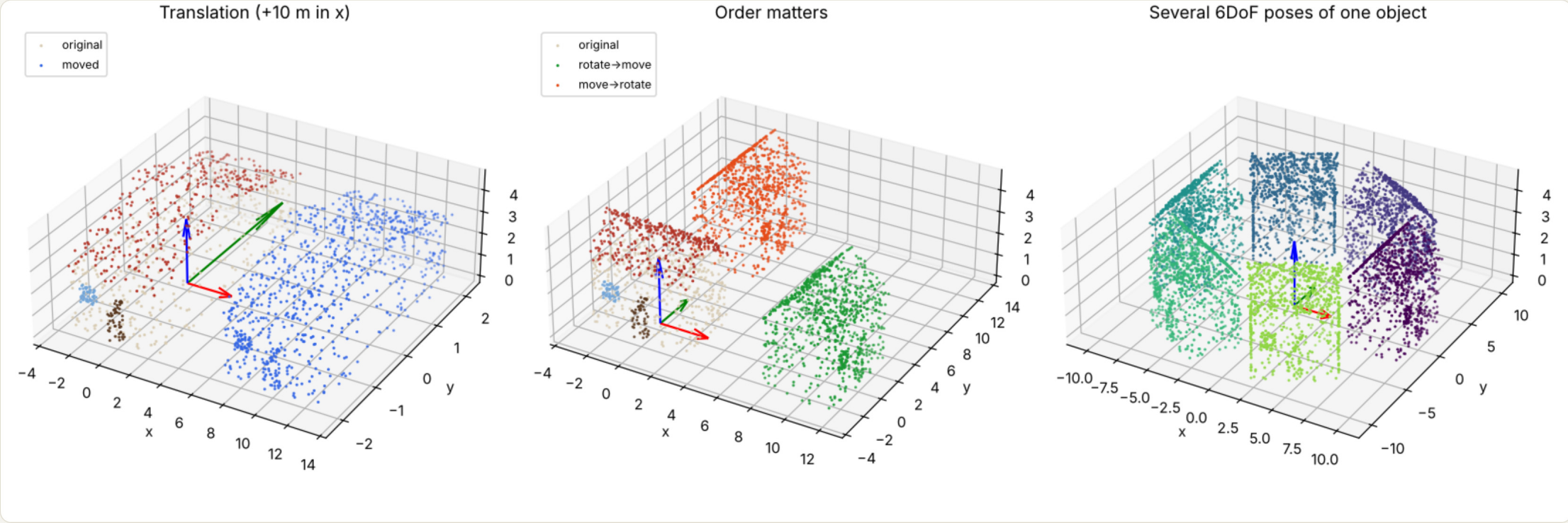
$T_{total} = T_3 \cdot T_2 \cdot T_1$ applies $T_1$ first.

### Undo

$T^{-1} = \begin{bmatrix} R^T & -R^T t \\ 0\ 0\ 0 & 1 \end{bmatrix}$

LAB PREVIEW

# Order matters: rotate → move ≠ move → rotate




From transforms.py. Middle: the same two operations in a different order land the house in different places.

THE 6DOF POSE

# 6 numbers → one matrix → 7 numbers on the wire

## Euler angles

yaw, pitch, roll. Easy for humans; suffers from gimbal lock and order ambiguity.

01

## Rotation matrix

9 numbers, no ambiguity, ideal to apply to points.

02

## Quaternion

4 numbers (w, x, y, z), smooth to interpolate. Engines and protocols send poses as position(3) + quaternion(4).

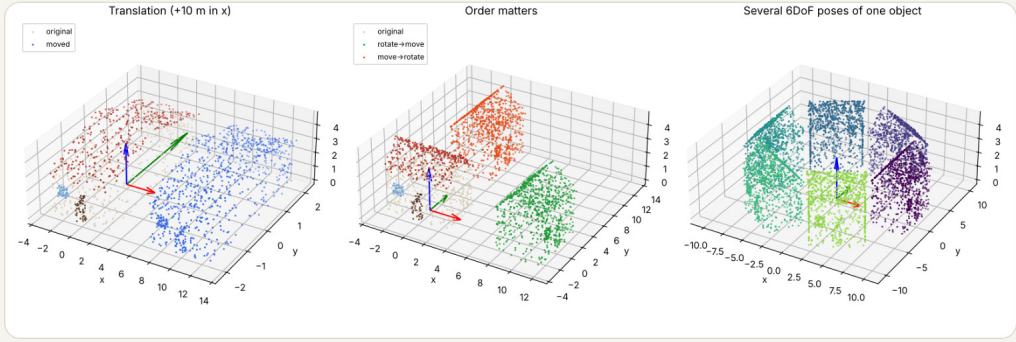
03

HANDS-ON LAB · COPY, RUN, LOOK

# Print every step of the maths

RUN IT

```
cd day02_coordinates_transforms
python transforms.py
```




→ Check R.T @ R = I and det(R) = 1 in the printout.

→ Compare the two centres for the two orders.

→ Find the quaternion line: 7 floats per pose.

# Three things to remember

1. Coordinates are relative to a frame. Know your axes: Z-up in Python, Y-up in three.js.
2. A 4×4 homogeneous matrix holds rotation + translation; chains of moves are one product.
3. A 6DoF pose travels as 7 floats: position + quaternion.

**BEFORE NEXT SESSION**

Finish Lab L1 (Days 1–2). Install the course environment if you haven’t: next session we write a full 3D program.